Nest JS Part.2

-2026S BE Seminar 2차시-

- By 박승범 (killerwhale) -

2-1. Env Vars

환경 변수

- 환경 변수(Environment Variable): 프로그램의 동작에 영향을 주는 변수
- 프로그램 자체를 수정하지 않고, 실행하는 환경에 따라 동작을 변경해야 할 때 사용
  - NODE_ENV: 개발 환경인지, 실제 배포 환경인지 구분하기 위함
  - 데이터베이스 연결 URL
  - 외부 API 비밀 키
  - 서버가 실행되는 포트(Port) 번호

  ...
- Node.js에서는 process.env를 통해 전달됨

```
35  export const env = envSchema.parse({
36    NODE_ENV: process.env.NODE_ENV,
37    PORT: process.env.PORT,
38    DATABASE_URL: process.env.DATABASE_URL,
39    MAX_UPLOAD_SIZE_MB: process.env.MAX_UPLOAD_SIZE_MB,
40    JWT_SECRET: process.env.JWT_SECRET,
41    JWT_EXPIRES_IN: process.env.JWT_EXPIRES_IN,
42    COOKIE_NAME: process.env.COOKIE_NAME,
43    ALLOWED_ORIGINS: process.env.ALLOWED_ORIGINS,
44    AWS_REGION: process.env.AWS_REGION,
45    AWS_S3_BUCKET: process.env.AWS_S3_BUCKET,
46    AWS_ACCESS_KEY_ID: process.env.AWS_ACCESS_KEY_ID,
47    AWS_SECRET_ACCESS_KEY: process.env.AWS_SECRET_ACCESS_KEY,
48    AWS_CLOUDFRONT_URL: process.env.AWS_CLOUDFRONT_URL,
49  });
```

.env 파일

- .env 파일: 텍스트 파일을 통해 환경 변수를 설정할 때 사용하는 사실상 표준 방식
  - 비밀 키와 같은 민감한 정보가 저장되므로,
    .gitignore 파일에 추가하여 **Git 레포지토리에 업로드되지 않도록** 하는 것이 일반적임
- Node.js에서는 전통적으로 dotenv 패키지를 사용해 해당 파일을 읽어왔으나,
  최신 버전의 Node.js에서는 dotenv 없이도 해당 파일을 읽어올 수 있음

```env
# General
PORT=8000

# Database
DATABASE_URL=mysql://root:root@localhost:3306/gallery   # Required
MAX_UPLOAD_SIZE_MB=10

# Security
JWT_SECRET=replace-with-long-random-secret              # Required
JWT_EXPIRES_IN=7d
COOKIE_NAME=auth
ALLOWED_ORIGINS=http://localhost:5173

# AWS
AWS_REGION=ap-northeast-2                               # Required
AWS_S3_BUCKET=sparcs-web-seminar                        # Required
AWS_ACCESS_KEY_ID=                                      # Required
AWS_SECRET_ACCESS_KEY=                                  # Required
AWS_CLOUDFRONT_URL=https://d5il24m7xaef1.cloudfront.net
```

```typescript
// 환경 변수에 따라 .env.production 또는 .env.development 파일을 읽어옵니다.
import dotenv from "dotenv";
import type { ServiceAccount } from "firebase-admin";
import type { Algorithm } from "jsonwebtoken";

if (process.env.NODE_ENV === undefined) {
  // logger.ts가 아직 초기화되지 않았으므로 console.error를 사용합니다.
  // eslint-disable-next-line no-console
  console.error("There is no NODE_ENV environment variable.");
  process.exit(1);
}
dotenv.config({ path: `./.env.${process.env.NODE_ENV}` });
```

2-2. Concurrency

Quiz

```javascript
console.log("1");

setTimeout(() => {
  console.log("2");
}, 2000);

const start = Date.now();
while (Date.now() - start < 4000) {
  // 4초 대기
}

console.log("3");
```

- 이 코드의 실행 결과는?

Quiz - Answer

```javascript
console.log("1");

setTimeout(() => {
  console.log("2");
}, 2000);

const start = Date.now();
while (Date.now() - start < 4000) {
  // 4초 대기
}

console.log("3");
```

- 이 코드의 실행 결과는?
- 1 (4초 후) 3 2
- Node.js에서는 가능하면 **비동기 함수**를 사용

Node.js

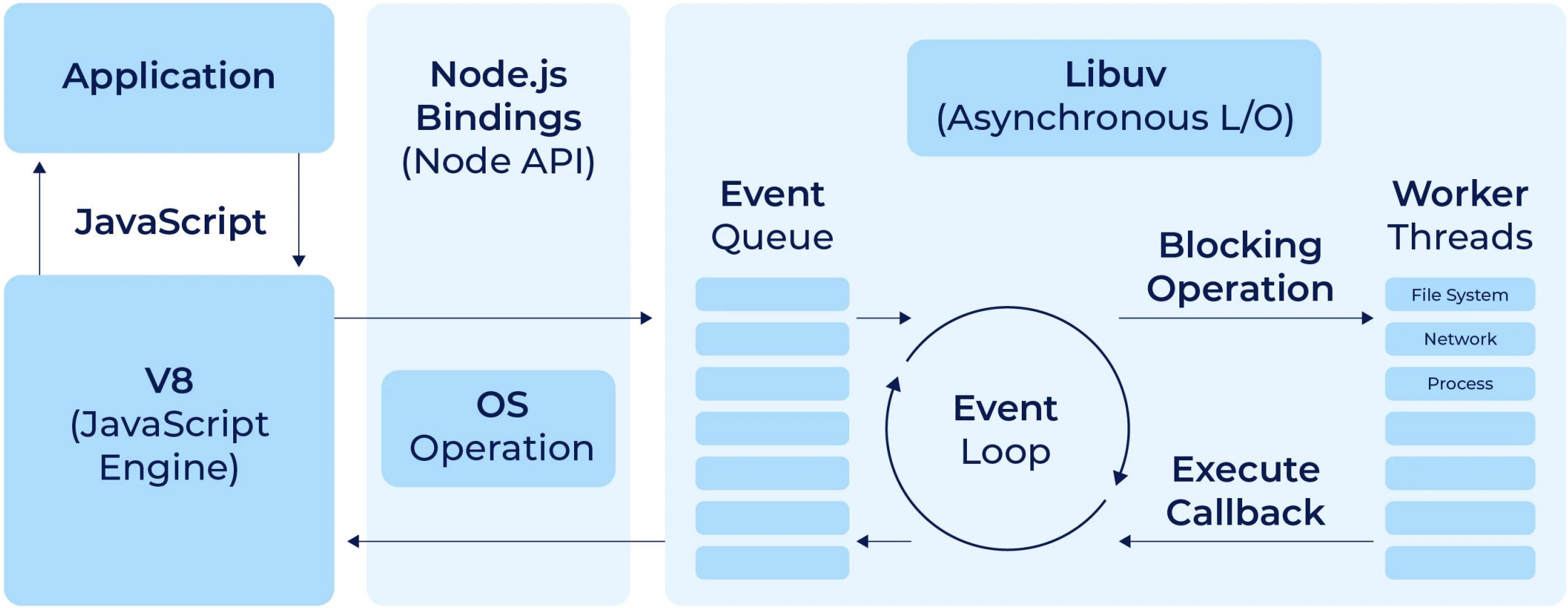
- "Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine."
- JavaScript 런타임: JavaScript 코드가 실행되는 환경
- JavaScript 엔진: JavaScript 코드의 인터프리터 + JIT 컴파일러

  ex) Google의 V8 엔진, 모질라 재단의 SpiderMonkey, ...
- 정리하면... Node.js는 웹 브라우저 밖에서도 JavaScript를 실행할 수 있도록 해주는 프로그램!

```
$ python3 main.py    # runs Python program
$ node index.js      # runs JavaScript program
```

- 비동기 I/O, 이벤트 기반 아키텍처 덕분에 효율적으로 웹 애플리케이션을 구동할 수 있음

Node.js


Application
JavaScript
V8
(JavaScript
Engine)
Node.js
Bindings
(Node API)
OS
Operation
Libuv
(Asynchronous I/O)
Event
Queue
Event
Loop
Blocking
Operation
Execute
Callback
Worker
Threads
File System
Network
Process

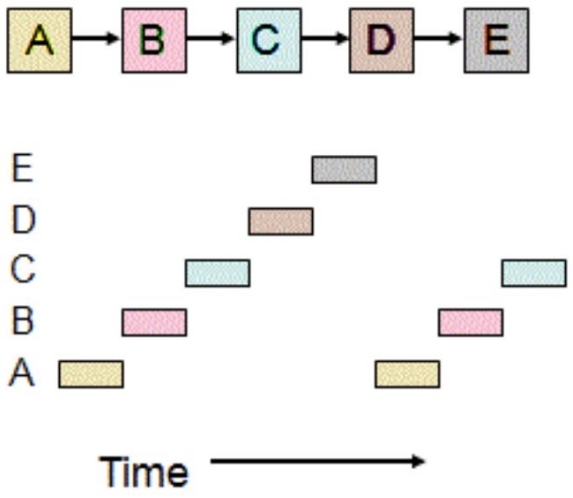
배경 지식: 스레드와 스케쥴링

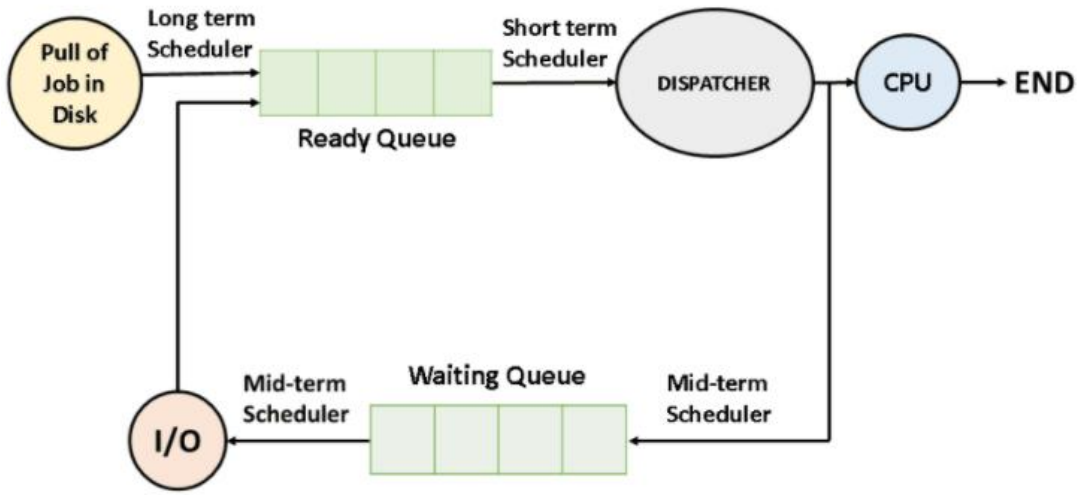
- CPU는 한 번에 하나의 명령어만 실행할 수 있음

  ...이 여러 의미로 사실이 아니라는 것을 CS311이나 EE312에서 배우게 됩니다.

- 하지만 실제로 우리는 CPU에서 여러 개의 프로그램을 동시에 실행하고 있음

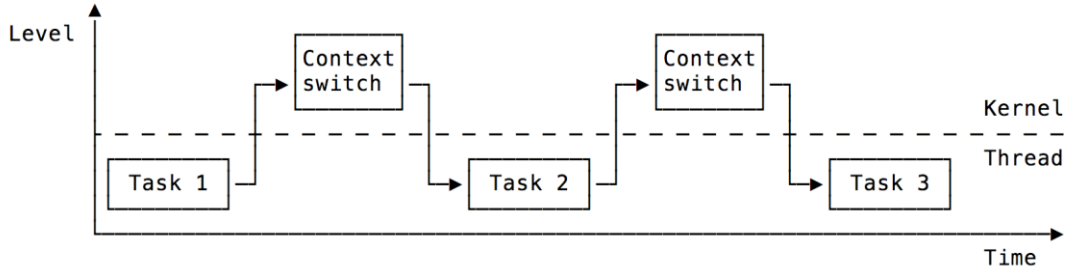
- 사실 동시에 실행되고 있는 것이 아니라, 동시에 실행되는 것처럼 보이는 것

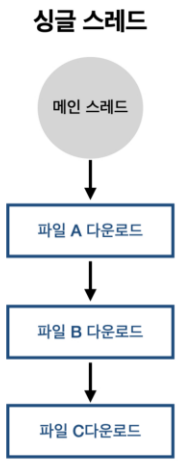


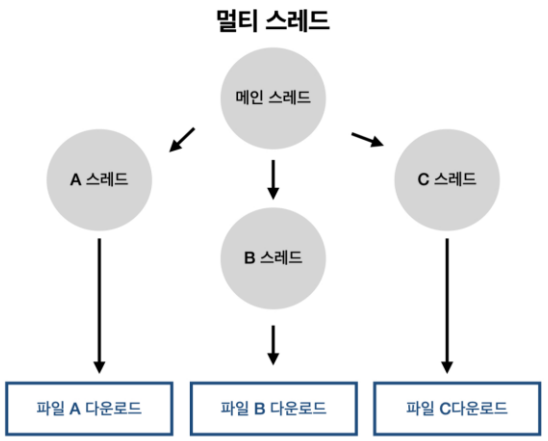


배경 지식: 스레드와 스케쥴링

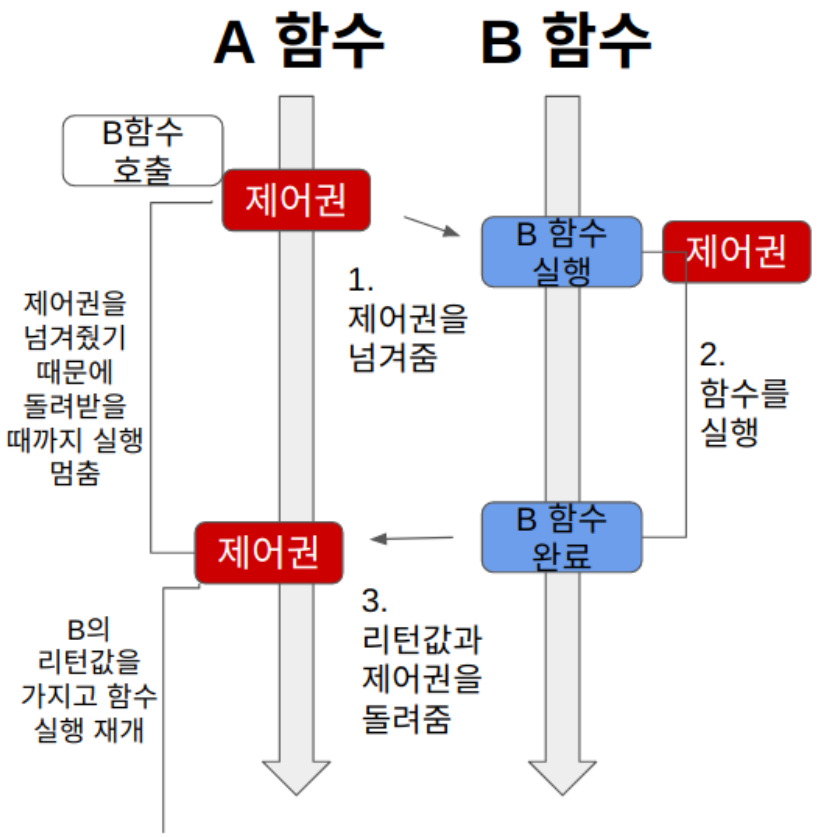
- 스레드(Thread): 실제로 명령어를 실행하는 주체
  - 하나의 프로그램(프로세스)은 여러 개의 스레드로 구성된다.
  - 스레드 내에서 명령어는 반드시 순서대로 실행된다.
- 스케쥴러(Scheduler): 스레드들을 규칙에 따라 적절히 중단, 실행시키며
  하나의 CPU에서 여러 개의 스레드가 효율적으로 실행될 수 있도록 관리하는 프로그램(?)
- JavaScript 코드의 실행은 싱글 스레드로 처리된다.



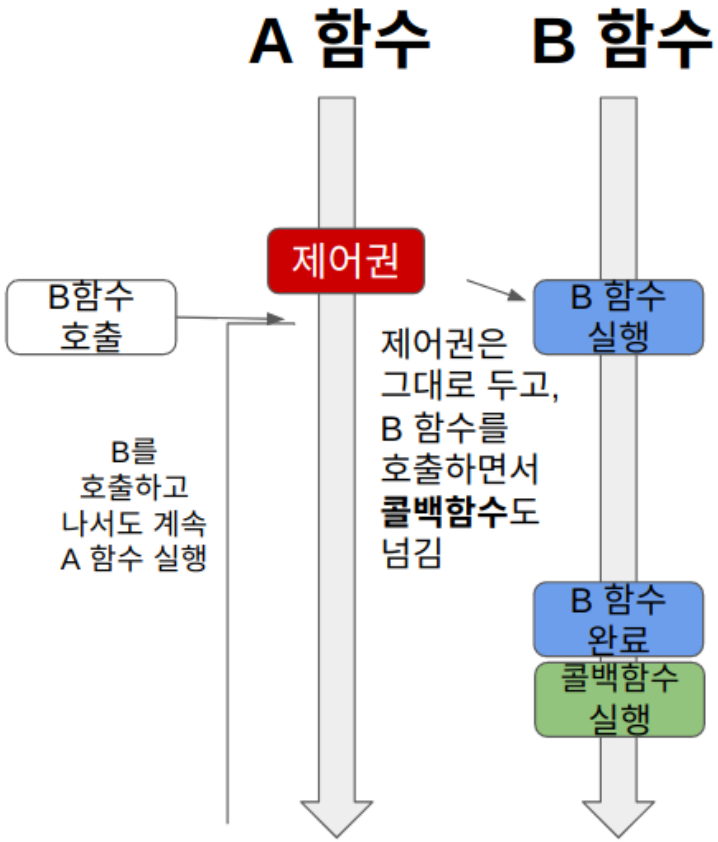




배경 지식: 동기 함수와 비동기 함수



동기-블로킹



비동기-논블로킹

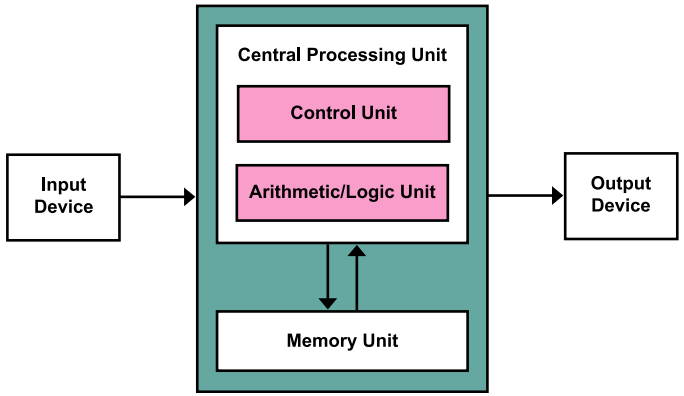
아이디어: CPU-Bound와 I/O-Bound

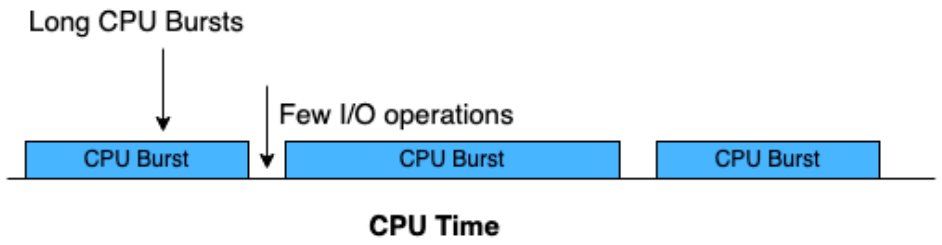
- 컴퓨터는 CPU와 입출력 장치로 구성됨
- CPU Burst: 연속적으로 CPU를 갈구는 것

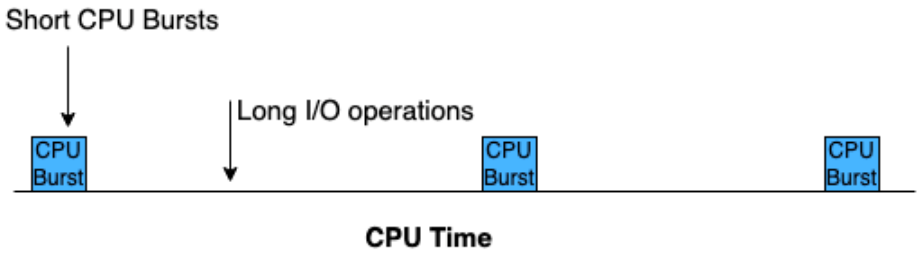
  I/O Burst: 스레드가 입출력 장치를 기다리는 것
- CPU-Bound: CPU Burst가 많은 경우

  I/O-Bound: I/O Burst가 많은 경우
- 대부분의 웹 백엔드는 I/O Bound
  - DB와의 통신, 클라이언트와의 네트워크 통신,

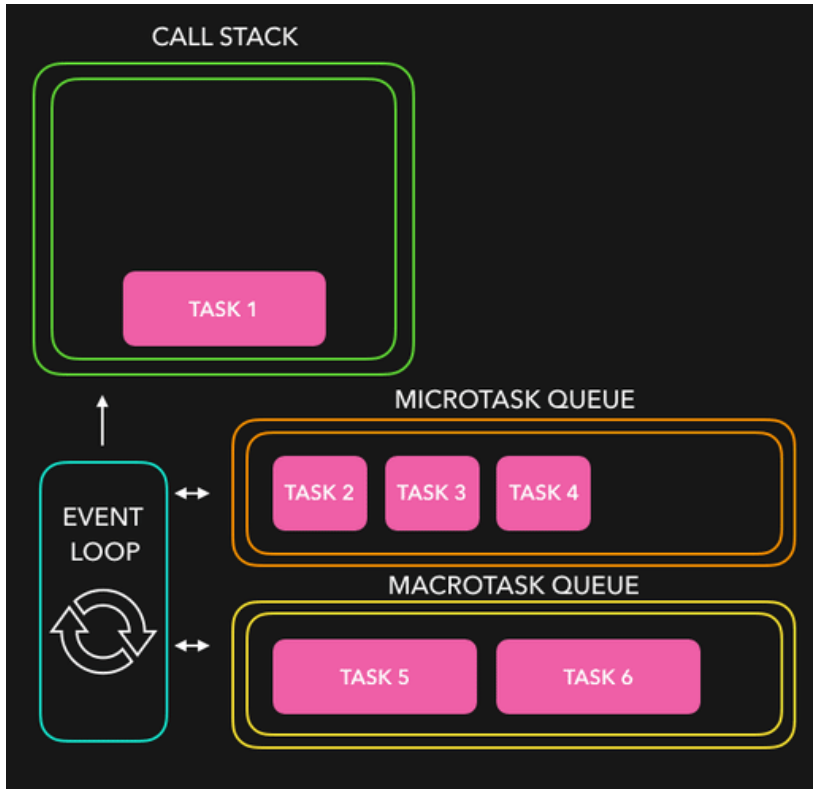
    로그 기록, 파일 읽기/쓰기, ...
- 결과를 기다리느라 CPU가 쉬는 꼴을 보고싶지 않다!







콜백 함수와 태스크 큐

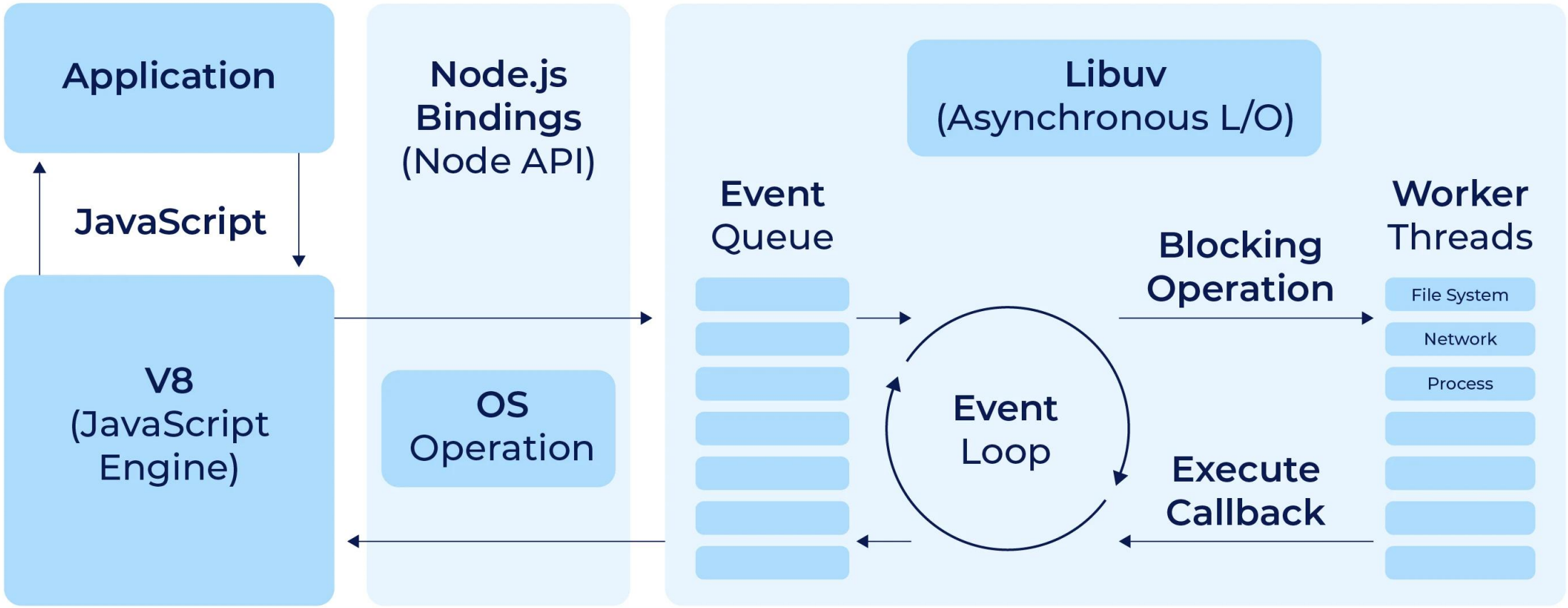


- I/O를 하는 동안에도 CPU를 갈구고 싶다.
- I/O가 끝난 후에만 실행할 수 있는 작업과,
  그렇지 않은 독립적인 작업이 있을 것이다.
  - DB에 쿼리를 날린 후 결과를 기다리고 있는 상황
  - **전자의 예:** DB로부터 결과를 받아 클라이언트에게 JSON으로 반환하기
  - **후자의 예:** 다른 HTTP 요청 처리하기
- '그렇지 않은 작업'을 먼저 처리하고, I/O가 끝나면 의존성이 있는 작업을 처리하면 어떨까?
- 콜백(Callback) 함수: 다른 함수에 전달된 후, 특정 조건이 만족되면 그때 호출되는 함수
- 태스크 큐: JavaScript 엔진이 앞으로 실행해야 할 콜백 함수를 쌓아두는 곳

Node.js 이벤트 루프

- **콜 스택**(Call Stack)**:** 실행 중인 함수의 정보를 저장하는 곳
- 콜 스택이 모두 비워지면, 즉 더이상 **실행할 동기 JavaScript 코드가 없으면**
  그 때부터 태스크 큐에 등록된 콜백 함수들이 순서대로 처리된다.
- Node.js에서는 작업에 따라 구분된 6가지 종류의 태스크 큐가 있어서,
  우선순위에 따라 효율적으로 콜백 함수를 처리해나간다.
- **이벤트 루프**
  : 콜스택이 비어있을 때, 태스크 큐에서 콜백 함수를 꺼내서 콜 스택에 집어넣는 매커니즘
- I/O 작업의 경우 Node.js 외부에서 처리되므로,
  I/O 작업이 처리되는 동안에도 JavaScript 코드를 실행하며 CPU를 최대한 갈굴 수 있다.

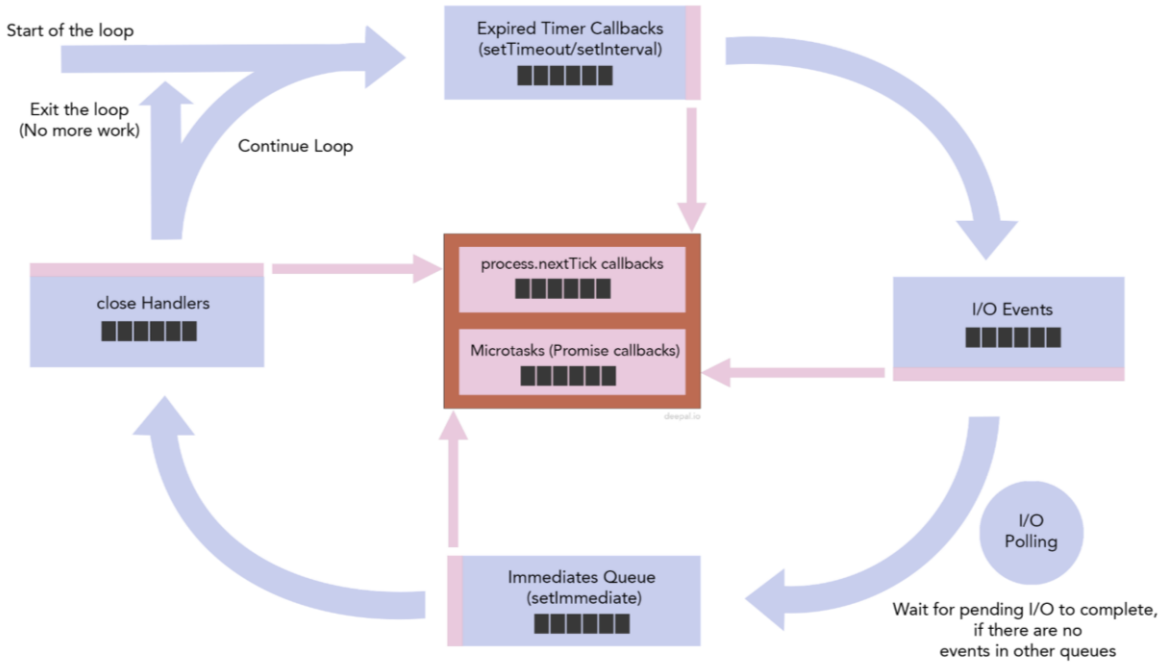
Node.js 이벤트 루프



참고: Phase와 Microtask

- 각 Phase마다 머무를 수 있는 시간이 정해져 있다.
  - 특정 Phase의 큐에 콜백 함수가 계속해서 들어오더라도, 그 Phase에 갖히지 않는다.
  - 다음 Phase로 넘어갈 때, Microtask들이 실행된다.
- **Timer Phase:** setTimeout() 등
- **Poll Phase**

  : DB, 네트워크 등 비동기 I/O와 관련됨
- **Microtask:** Promise.then() 등



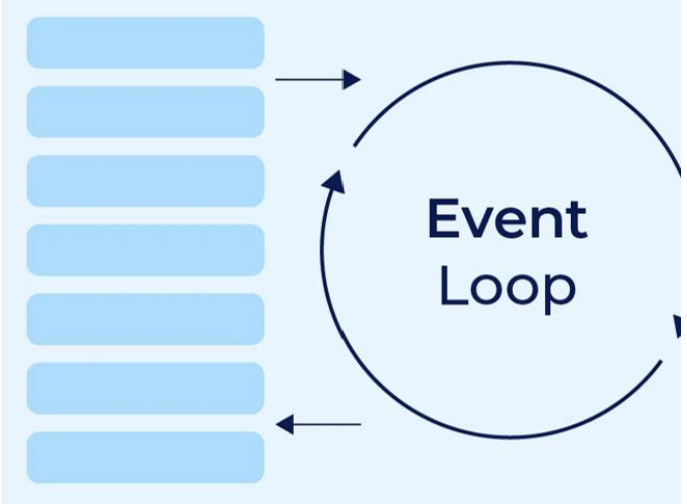
Race Condition

2개 이상의 thread가 동일한 resource에 접근할 때 발생하는 정합성 문제

```
@Injectable()
export class PointService {
  constructor(private readonly userRepository: Repository<User>) {}

  // 사용자의 포인트를 10만큼 차감하는 비즈니스 로직
  async usePoints(userId: number) {
    // 1. 현재 사용자의 데이터(포인트 잔액)를 DB에서 조회
    const user = await this.userRepository.findOne({ where: { id: userId } });

    // 2. 잔액 검증
    if (user.points >= 10) {
      // 3. 메모리 상에서 포인트 차감
      user.points -= 10;

      // 4. 변경된 값을 DB에 저장 (UPDATE)
      await this.userRepository.save(user);
      return true;
    }
    throw new Error('포인트가 부족합니다.');
  }
}
```

이 코드의 문제점은?

Race Condition



T1 : 남은 재고 몇 개야?

DB : 1개

T2 : 남은 재고 몇 개야?

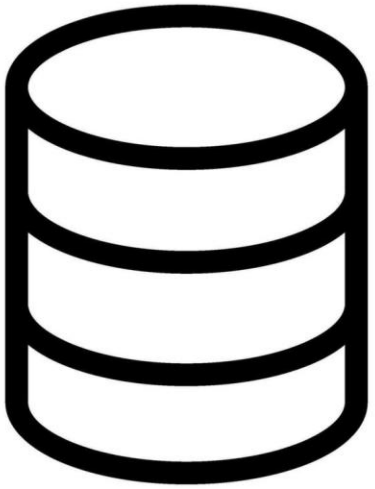
DB : 1개

T1 : 그럼 1개 줘.

DB : ok

T2 : 그럼 1개 줘.

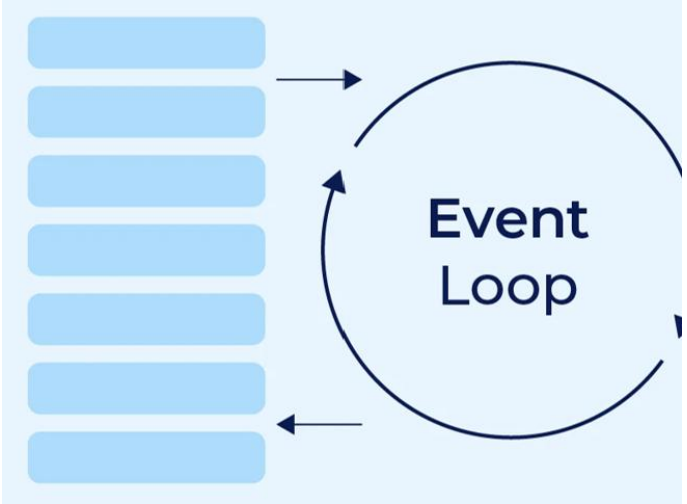
DB : ???



Solution – Transaction Lock

정합성을 유지해야하는 연산을 Transaction화

한 번에 하나의 Transaction만 DB에 접근이 가능하도록 Lock 설정

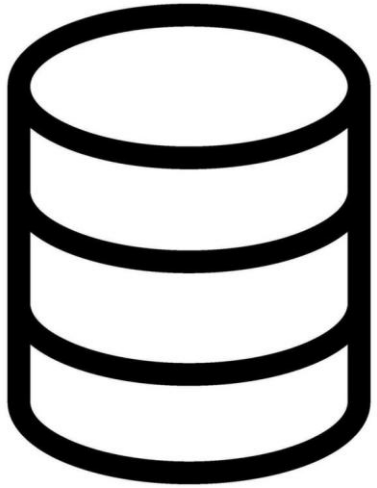


T1 : 남은 재고 몇 개야? 있으면 1개 줘

DB : ok

T2 : 남은 재고 몇 개야? 있으면 1개 줘.

DB : 남은 재고 0개야.



Race Condition in NestJS

NestJS의 Dependency Injection -> 프레임워크가 인스턴스 관리

효율을 위해 하나의 인스턴스를 재사용 하는 Singleton (싱글턴) 패턴

```
@Injectable()
export class OrderService {
  private userId: string; // 공유 자원 (싱글톤 인스턴스 변수)

  async process(id: string) {
    this.userId = id;
    await this.pay(); // 컨텍스트 스위칭 (제어권 양보 및 대기)
    return `Receipt for: ${this.userId}`; // 타 요청에 의한 데이터 오염 발생
  }
}
```

요청은 A-> B로 들어왔지만

Pay()가 B -> A 순서로 끝난다면?

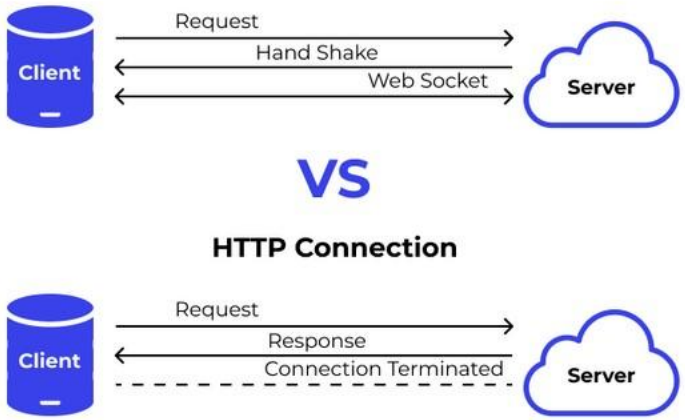
-> A에게 B의 정보가 갈 수 있음!

2-3. Socket

Socket

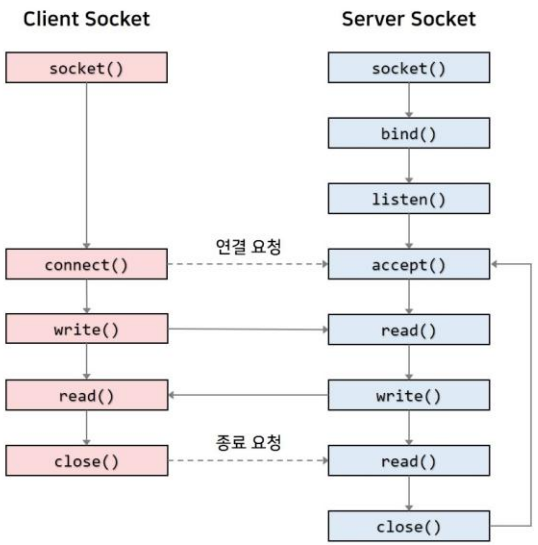
Socket vs. HTTP

- 한번 connection이 수립되면, 양쪽에서 언제든 통신 가능.
- 채팅, 주식 등 실시간성이 필요한 통신에 활용



Socket in NestJS

- Socket.io / ws 라이브러리를 활용
- **Controller 대신 Gateway를 활용**



More Detail on CS230 and CS341

Socket - Decorators

@WebSocketGateway

- 소켓 서버 역할을 하는 클래스 정의를 위해 활용 (Controller와 대응)

@WebSocketServer

- 실행중인 socket 인스턴스 -> BroadCasting 등에 활용

@SubscribeMessage()

- 특정 이벤트 감지. API 엔드포인트에 대응

```
import { WebSocketGateway, SubscribeMessage, MessageBody, WebSocketServer, Co
import { Server, Socket } from 'socket.io';

@WebSocketGateway({ cors: true }) // 기본 포트 공유, CORS 허용
export class ChatGateway {

  // 현재 동작 중인 웹소켓 서버 인스턴스를 주입받음
  @WebSocketServer()
  server: Server;

  // 클라이언트가 'sendMessage'라는 이름으로 보낸 이벤트를 수신
  @SubscribeMessage('sendMessage')
  handleMessage(
    @ConnectedSocket() client: Socket, // 메시지를 보낸 클라이언트의 소켓 객체
    @MessageBody() payload: { user: string; text: string } // 전송된 데이터
  ) {
    // 수신한 메시지를 서버에 연결된 '모든' 클라이언트에게 전달 (Broadcast)
    this.server.emit('receiveMessage', payload);
  }
}
```

Socket – DI & LifeCycle

GateWay도 Controller 처럼 DI를 받을 수 있다.

-Service를 Consturtor를 통해 DI를 받아 편리하게 관리 가능!

HTTP와 달리 Lifecycle Hook 존재

- HTTP와 다르게 클라이언트의 접속/접속 종료 상태를 관리
- OnGatewayConnection, OnGatewayDisconnect 등의 훅 사용

```
@WebSocketGateway()
export class ChatGateway implements OnGatewayConnection, OnGatewayDisconnect

  // 비즈니스 로직(예: 채팅 내역 DB 저장)을 담당하는 Service를 주입받음
  constructor(private readonly chatService: ChatService) {}

  // 클라이언트가 소켓 서버에 연결되었을 때 실행
  handleConnection(client: Socket) {
    console.log(`Client connected: ${client.id}`);
    // 필요 시 사용자 인증 로직을 여기서 수행할 수도 있음
  }

  // 클라이언트가 통신을 끊고 나갔을 때 실행
  handleDisconnect(client: Socket) {
    console.log(`Client disconnected: ${client.id}`);
  }

  @SubscribeMessage('sendMessage')
  async handleMessage(@MessageBody() payload: any) {
    // 1. DB에 채팅 기록 저장 (Service에 위임)
    const savedMessage = await this.chatService.saveMessage(payload);

    // 2. 다른 사용자들에게 브로드캐스팅
    this.server.emit('receiveMessage', savedMessage);
  }
}
```

2-4. Query Optimizatoin

N + 1 쿼리

1개의 쿼리로 list N개를 가져온 뒤 각 항목과 연관된 데이터를 얻기 위해

N번의 추가 쿼리가 발생하는 현상

Ex) 게시판에서 글 10개 조회 이후 작성자를 알기 위해 user table 10번 조회

Solution : 연관된 Table을 Join 하여 1번의 쿼리로 가져오기

-Prisma ORM 에서는 include

```
// Bad: N+1 문제 발생 (게시글 목록 조회 1번 + 루프를 돌며 작성자 조회 N번)
const articles = await prisma.article.findMany();
for (const article of articles) {
  article.author = await prisma.user.findUnique({
    where: { id: article.authorId }
  });
}

// Good: include 옵션을 사용하여 연관 데이터를 한 번에 가져옴 (N+1 방지)
const articles = await prisma.article.findMany({
  include: { author: true },
});
```

Composite Index

*레포지토리에 where 절을 쓸 때 모든 변수를 포함하는 composite index가 있는지 확인하기!!

Composite Index (복합 인덱스)

- 여러 개의 컬럼을 조합하여 DB를 정렬 (index를 생성) 하는 것
- 보통 where절에 걸려있는 변수들을 묶어서 복합 인덱스를 생성한다.

복합 인덱스 순서

- 복합인덱스는 왼쪽부터 matching된다.
- **Cardinality : 가장 많은 데이터를 filtering 할 수 있는 index를 왼쪽에 걸어야 빨라진다.**

Ex: (name, age)로 걸려 있다면 name이 순서대로, name이 같다면 age순서대로 정렬되어 있다.

Ex: (name, age)인덱스만 있다면 where 절에 age 만 있을 때 index를 타지 못 함

Index Miss

Explain (쿼리 내용)

Explain Analyze (쿼리 내용)

을 쿼리하여 SQL 옵티마이저가 어떤 index를 선택하는지 볼 수 있다.

SQL Optimizer

- 쿼리를 분석하여 어떤 index를 기준으로 Search 해야 하는지 자동으로 선택한다.
- 잘못 분석할 경우 DB 쿼리가 매우 느려지는 문제가 발생 (O(log n) 이 O(n)이 된다) (대부분 큰 문제 없다.)
- 수동으로 경로를 지정할 수 있지만, Prisma, Django 등의 ORM에서 인덱스 설정을 지원하지 않는다.

애초에 잘못 분석되지 않도록 쿼리를 설계해야 한다.

Index Miss 방지하기

게시글 조회 쿼리에서 게시글을 조회하고 subquery로 댓글을 조회하는 것 보다 두 개의 쿼리로 나누는 것이 유리하다.

- 조건 단순화 (index는 범위 조건 a < x < b 에 취약하다)
- **서브쿼리 제거 -> Nested Object Prefetching (많은 Table을 한 쿼리에 Join 하는 것을 지양)**
- **원본 컬럼 유지**

```sql
-- Bad: 인덱스 컬럼을 함수로 가공 (Index Miss 발생)
SELECT * FROM users WHERE SUBSTRING(student_id, 1, 4) = '2025';

-- Good: 컬럼 원본 유지 (LIKE 패턴 매칭 또는 범위 검색 활용)
SELECT * FROM users WHERE student_id LIKE '2025%';
```

Thanks for Attention!

도움을 주신 모든 분들께 감사 드립니다.

- Special thanks for @static

KAIST SPARCS